



ORGANIZACIÓN Y BÚSQUEDA DE PRODUCTOS EN UNA TIENDA

PRESENTADO POR:

RAFAEL.HENAO

THOMAS.PEÑA

JORGE.SILVA

INTRODUCCION AL PROBLEMA

- Contexto del Negocio / Aplicación:
- En sistemas de inventario y gestión de productos, las empresas manejan listas dinámicas de elementos identificados por códigos numéricos.
- El Reto:
- Realizar búsquedas en listas desordenadas requiere revisar elemento por elemento (búsqueda lineal), lo cual es ineficiente cuando el volumen de datos crece ($O(n)$).
- La Solución Propuesta:
- Ordenar los productos utilizando un algoritmo eficiente de división y conquista (Merge Sort) para luego ejecutar búsquedas rápidas en tiempo logarítmico mediante Búsqueda Binaria ($O(\log n)$).

FUNCIONES DEL CODIGO

- `Merge_sort(codes):`
- **Propósito:** Función recursiva que divide la lista de códigos por la mitad hasta obtener sublistas de 1 o 0 elementos (casos base). Luego llama a merge para combinarlas en orden.
- `Merge(izq_ord, der_ord):`
- **Propósito:** Función auxiliar que recibe dos sublistas ya ordenadas y las fusiona elemento por elemento en una nueva lista en orden ascendente.
- `Bus_bin(codes, obj):`
- **Propósito:** Realiza la búsqueda de un producto objetivo (obj) en una lista previamente ordenada, reduciendo el área de búsqueda a la mitad en cada iteración.
- `Main():`
- **Propósito:** Función principal que interactúa con el usuario, solicita y valida los datos de entrada, coordina las llamadas a los algoritmos y muestra el resultado final.

PASOS DEL CODIGO BASE DEL 1 AL 5

- **Paso 1: Solicitud e Validación de Cantidad (n)**El programa pide ingresar cuántos productos se van a registrar. Valida que $n > 0$; si no se cumple, detiene el flujo con un mensaje de advertencia.
- **Paso 2: Captura y Validación de Códigos**Mediante un ciclo for, se le pide al usuario ingresar cada código de producto. Valida que ningún código sea negativo ($temp < 0$).
- **Paso 3: Ordenamiento con Merge Sort**Se invoca `merge_sort(codes)`. El arreglo original se divide iterativamente a la mitad y se reconstruye de menor a mayor mediante `merge()`, devolviendo `codes_ord`.
- **Paso 4: Solicitud del Producto a Buscar**Se pide el código objetivo (`obj`) y se valida que no sea un número negativo.
- **Paso 5: Búsqueda y Muestra del Resultado**
- Se ejecuta `bus_bin(codes_ord, obj)`. Si el índice retornado es distinto de -1, se confirma que el producto existe; de lo contrario, se notifica que no se encuentra registrado.

BÚSQUEDA EFICIENTE DE FUNCION BINARIA

- Mecanismo de Funcionamiento:
- Define dos punteros: $ini = 0$ (inicio) y $fin = len(codes) - 1$ (final). Calcula el punto medio con división entera: $med = (ini + fin) // 2$.
- Evaluación de condiciones: Si $codes[med] == obj$: Elemento encontrado (devuelve la posición med). Si $codes[med] < obj$: El elemento buscado es mayor, por lo que se descarta la mitad izquierda ajustando $ini = med + 1$. Si $codes[med] > obj$:
- El elemento buscado es menor, por lo que se descarta la mitad derecha ajustando $fin = med - 1$. Si ini supera a fin , se concluye que el elemento no está en la lista y retorna -1. Ventaja
- Algorítmica: Reduce la complejidad de búsqueda a $O(\log n)$, lo cual es significativamente más rápido que comparar elemento por elemento.

CONCLUSIONES Y ERRORES DE SISTEMA

- **Control de Errores e Invariantes:** Validación de entradas: El sistema previene comportamientos inesperados comprobando que el número de productos sea mayor que 0 y que los códigos no sean negativos. Manejo de estados no encontrados: Retornar -1 en la búsqueda binaria permite separar la lógica de cálculo de la lógica de visualización al usuario.
- **Conclusiones Técnicas:** Interdependencia de algoritmos: La Búsqueda Binaria requiere estrictamente que los datos estén ordenados previamente. Escalabilidad: La combinación de Merge Sort ($O(n \log n)$) y Búsqueda Binaria ($O(\log n)$) garantiza un rendimiento óptimo incluso cuando el inventario escala a miles de productos.